SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA	**Total Marks:**	100
CO1 Statement:	*Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)*		
Student Name:	M Rakesh	**Reg. No.:**	192421006
Section / Batch:	IT	**Date:**	23/8/26

Instructions to Students

- Answer the following question. It carries 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical explanation are mandatory

Question Type	Focus Area	Questions
Application-Based Questions	Apply Master Theorem and asymptotic analysis to real-world scenarios	Q1

SECTION A – APPLICATION BASED QUESTIONS

Q1. An insurance verification system uses multiple nested loops to validate policy details, customer history, and claim records simultaneously. Explain how nested loops contribute to higher time complexity and describe the resulting growth rate. Determine the asymptotic time complexity using Big-O notation and justify your reasoning clearly. Analyze the space complexity, including memory used for storing intermediate validation results. Interpret how this approach performs for large datasets and discuss its efficiency limitations.

Code of Conduct

I _____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%				
Application of Concepts	30%				
Problem-Solving Approach	20%				
Accuracy of Solution	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1. An Insurance Verification System Using Multiple Nested Loops

Introduction

An insurance verification system checks several types of information before approving a customer's claim. The system verifies policy details, customer history, and claim records to ensure that all the information is correct. One simple way to perform these checks is by using multiple nested loops. Although this method is easy to understand and implement, it performs a large number of comparisons when the dataset grows, making it less efficient for large applications.

Working of Nested Loops

Assume that there are **n** policy records, **n** customer records, and **n** claim records. The verification process may use three nested loops as shown below.

```
for(i = 1; i <= n; i++)
    {
       for(j = 1; j <= n; j++)
       {
          for(k = 1; k <= n; k++)
          {
             Verify Policy, Customer and Claim;
          }
       }
    }
```

The outer loop executes **n** times. For every iteration of the outer loop, the middle loop also executes **n** times. Similarly, the inner loop executes **n** times for every iteration of the middle loop. Therefore, the total number of operations performed is:

$\mathbf{n \times n \times n = n^3}$

This shows that the number of comparisons increases rapidly as the number of records increases.

Time Complexity Analysis

Time complexity represents the amount of time required by an algorithm as the input size increases. For the above algorithm,

- Outer loop = **n** iterations
- Middle loop = **n** iterations
- Inner loop = **n** iterations

Therefore,

$\mathbf{T(n) = n \times n \times n = n^3}$

Ignoring constants and lower-order terms, the asymptotic time complexity is:

Time Complexity = $O(n^3)$

Justification

Big-O notation represents the worst-case performance of an algorithm. Since all three loops execute completely regardless of the input values, the total number of operations is proportional to $\mathbf{n^3}$.

For example:

- If **n = 10**, operations ≈ **1,000**
- If **n = 100**, operations ≈ **1,000,000**
- If **n = 1000**, operations ≈ **1,000,000,000**

This clearly shows that the execution time increases very quickly as the input size grows.

Space Complexity

Space complexity refers to the amount of memory required during execution. The algorithm uses memory for:

- Loop variables
- Temporary variables
- Intermediate validation results

If the system stores intermediate validation results in an array or list of size **n**, the extra memory required is proportional to **n**.

Therefore,

Space Complexity = O(n)

If no intermediate results are stored and only loop variables are used, then the space complexity becomes:

Space Complexity = O(1)

Growth Rate

The algorithm has **cubic growth** because its running time increases in proportion to $\mathbf{n^3}$.

Number of Records (n)	Approximate Operations
10	1,000
50	125,000
100	1,000,000
500	125,000,000
1000	1,000,000,000

From the table, it is clear that even a small increase in the input size causes a very large increase in execution time.

Performance for Large Datasets

For a small insurance company with only a few records, this algorithm works satisfactorily. However, modern insurance companies maintain millions of customer and claim records. In such cases, the cubic growth results in a huge number of comparisons, increasing execution time and CPU usage. This makes the algorithm unsuitable for real-time applications and large databases.

Efficiency Limitations

The limitations of this approach are:

- Execution time increases rapidly as the input size grows.
- Performs many unnecessary comparisons.
- Consumes more processing time for large datasets.
- Does not scale well for enterprise-level applications.
- Reduces the overall performance of the system.

To overcome these limitations, practical systems use optimized database queries, indexing, hashing, and efficient search algorithms instead of multiple nested loops.

Real-World Applications

Nested loops are commonly used in situations where every record must be compared with other related records. Some real-world applications include:

- Insurance companies verify policy details, customer history, and claim records before approving claims.
- Banks compare transaction records to identify duplicate or fraudulent transactions.
- Hospital management systems verify patient records with prescriptions and treatment history.
- E-commerce websites compare orders, products, and customer details during order processing.
- Database management systems use nested iterations while performing certain join operations.

Conclusion

Multiple nested loops are simple to implement and understand, but they are computationally expensive. Since each loop executes **n** times, the overall time complexity becomes **$O(n^3)$**. The space complexity is **O(n)** if intermediate validation results are stored, otherwise it is **O(1)**. Although this approach works well for small datasets, it becomes inefficient for large databases because of its cubic growth rate. Therefore, optimized algorithms are preferred in real-world insurance systems.

Q2. Transaction System Using the Recurrence $T(n) = T(n - 1) + \log n$

Introduction

A transaction processing system performs several operations while processing records. The execution time

can be represented by the recurrence relation:

T(n) = T(n − 1) + log n

This means that the algorithm first processes a problem of size **n − 1** and then performs an additional **log n** amount of work. The substitution method is used to solve this recurrence and determine its time complexity.

Solving the Recurrence Using the Substitution Method

Given,

T(n) = T(n − 1) + log n

Expanding the recurrence,

T(n) = T(n − 2) + log(n − 1) + log n

Again,

T(n) = T(n − 3) + log(n − 2) + log(n − 1) + log n

Continuing the expansion,

T(n) = T(1) + log 2 + log 3 + ... + log n

This can be written as:

T(n) = T(1) + Σ log i, where **i = 2 to n**

Using the logarithm property,

Σ log i = log(n!)

Using Stirling's approximation,

log(n!) = Θ(n log n)

Therefore,

T(n) = Θ(n log n)

Hence,

Time Complexity = O(n log n)

Explanation

The algorithm performs one recursive call for each value from **n** down to **1**. At every step, only **log n** additional work is done. Adding these logarithmic values results in **Θ(n log n)**, making the algorithm much more efficient than quadratic or cubic algorithms.

Space Complexity

When recursion is used, each recursive call remains on the call stack until execution is complete.

Therefore,

Space Complexity = O(n)

If the same algorithm is implemented iteratively, only a few variables are required.

Hence,

Space Complexity = O(1)

Growth Behaviour

The growth rate of **O(n log n)** is between **O(n)** and **O(n²)**. It increases faster than a linear algorithm but much slower than quadratic and cubic algorithms. This makes it suitable for handling moderately large datasets efficiently.

Advantages

- Better performance than **O(n²)** and **O(n³)** algorithms.
- Suitable for moderately large datasets.
- Uses less execution time than many brute-force approaches.
- Commonly found in efficient algorithms used in software development.

Limitations

- Recursive implementation requires additional stack memory.
- Slightly slower than a purely linear algorithm.
- Execution time still increases with very large datasets.

Real-World Applications

Recurrence relations with **O(n log n)** complexity are used in many practical applications, such as:

- Merge Sort and other efficient sorting algorithms.
- Database indexing and searching operations.
- File system traversal and directory processing.
- Banking and e-commerce transaction processing systems.
- Data analysis applications that process large datasets recursively.

Conclusion

The recurrence relation **T(n) = T(n − 1) + log n** is solved using the substitution method by repeatedly expanding the recurrence until the base case is reached. The resulting expression becomes **log(n!)**, which simplifies to **Θ(n log n)** using Stirling's approximation. Therefore, the time complexity is **O(n log n)**. The space complexity is **O(n)** for the recursive implementation and **O(1)** for the iterative implementation. This complexity is efficient for many real-world applications because it scales much better than quadratic and cubic algorithms while maintaining good performance for large datasets.